

MEGA PROMPT — TRANSFORMAR STOCKDESK EN PRODUCTO COMERCIAL LISTO PARA PRODUCCIÓN

0. IDENTIDAD DEL PROYECTO

Proyecto: **StockDesk**

Repositorio: <https://github.com/Junmoxia41/StockDesk>

Propietario/desarrollador: **Junmoxia41**

Objetivo final:

Convertir el proyecto actual de StockDesk en un **producto de software profesional, estable, seguro, documentado, mantenible, comercializable y listo para producción**, preparado para:

- despliegue en Vercel;
- presentación pública;
- demostración comercial;
- distribución a clientes;
- venta de licencias;
- futuras actualizaciones;
- soporte técnico;
- mantenimiento;
- escalabilidad;
- protección razonable de la propiedad intelectual;
- uso real por pequeños y medianos negocios.

NO quiero que simplemente describas el proyecto.

Quiero que actúes como:

- Senior Software Architect
- Senior Frontend Engineer
- QA Engineer
- Application Security Engineer
- DevOps Engineer
- Technical Writer
- Product Manager
- UX/UI reviewer
- Release Engineer
- Software Licensing Consultant

Tu trabajo consiste en **AUDITAR, CORREGIR, ORGANIZAR, PROFESIONALIZAR, DOCUMENTAR Y PREPARAR StockDesk PARA SU VENTA.**

1. REGLA PRINCIPAL

NO DES POR HECHO QUE UNA FUNCIÓN EXISTE SOLO PORQUE ESTÁ DOCUMENTADA.

Debes verificarla en el código.

Para cada funcionalidad:

1. encontrar dónde está implementada;
2. analizar el flujo;
3. comprobar dependencias;
4. comprobar errores;
5. comprobar persistencia;
6. comprobar interacción con otras funciones;
7. determinar si realmente funciona;
8. determinar si funciona parcialmente;
9. determinar si es simulación;
10. determinar si está incompleta;
11. corregirla cuando sea posible.

Clasifica cada función como:

-  PRODUCCIÓN
-  FUNCIONAL PERO MEJORABLE
-  PARCIAL
-  ROTA
-  SIMULADA
-  NO IMPLEMENTADA

Nunca presentes como funcionalidad real algo que solamente sea una simulación.

2. PRIMERA FASE — AUDITORÍA COMPLETA

Antes de modificar demasiado código, realiza una auditoría completa.

Analiza:

- estructura del proyecto;
- HTML;
- CSS;
- JavaScript;
- módulos;
- páginas;
- dependencias;
- CDN;
- APIs externas;
- almacenamiento local;
- autenticación;

- seguridad;
- IA;
- impresión;
- exportaciones;
- generación PDF;
- backups;
- importación/exportación;
- rutas;
- navegación;
- responsive;
- accesibilidad;
- rendimiento;
- errores JavaScript;
- compatibilidad navegador;
- Vercel;
- PWA;
- service worker;
- manifest;
- caché;
- offline;
- persistencia;
- manejo de errores;
- validaciones;
- integridad de datos.

Busca especialmente:

- código duplicado;
- funciones gigantes;
- funciones muertas;
- variables globales;
- dependencias implícitas;
- módulos acoplados;
- nombres inconsistentes;
- archivos sin usar;
- imports inexistentes;
- referencias rotas;
- IDs duplicados;
- listeners duplicados;
- memory leaks;
- XSS;
- inyección;
- almacenamiento inseguro;
- secretos expuestos;
- API keys en frontend;
- datos sensibles almacenados sin protección;
- rutas que pueden romperse al desplegar;
- problemas de hash routing;
- errores al recargar páginas;
- problemas de Safari/iOS;
- problemas de dispositivos móviles.

3. INSPECCIÓN DEL REPOSITORIO

Debes recorrer TODO el repositorio.

No analices únicamente:

- index.html
- app.js
- README

Analiza todos los archivos.

Incluye:

`/css /docs /js /js/pages /js/modules`

y cualquier otro archivo que aparezca posteriormente.

Obtén un inventario de:

- archivos;
- líneas aproximadas;
- propósito;
- dependencias;
- relación entre módulos;
- funciones principales.

Genera un mapa de arquitectura.

Ejemplo conceptual:

UI ↓ Router ↓ Pages ↓ Components ↓ Store ↓ Modules ↓ External APIs

Corrige la arquitectura si está mal definida.

4. REGLA DE TAMAÑO DE ARCHIVOS

Quiero mantener el código modular.

OBJETIVO:

ningún archivo JavaScript debería superar aproximadamente 200 líneas cuando sea razonable dividirlo.

Si un archivo es demasiado grande:

- dividirlo;
- mantener responsabilidades claras;
- evitar duplicación;
- mantener APIs internas coherentes;
- evitar romper funcionalidades existentes.

No hagas divisiones arbitrarias.

Divide por responsabilidad.

5. ARQUITECTURA PROFESIONAL

Refactoriza StockDesk hacia una arquitectura clara.

Como mínimo separar conceptualmente:

Core

- app
- router
- store
- configuration
- utilities
- event bus si realmente resulta necesario

UI

- components
- layout
- modal
- toast
- forms
- tables
- loading states

Pages

- dashboard
- products
- sales
- reports
- inventory
- finance
- security
- users
- notifications
- suppliers
- customization

Business Modules

- inventory
- finance
- security
- AI
- suppliers
- notifications
- users

Services

- storage
- backup
- export
- PDF
- printing
- AI provider
- authentication

Evita que una página contenga directamente toda la lógica de negocio.

6. AUDITORÍA DEL STORE

Analiza profundamente `store.js`.

Determina:

- cómo se almacenan los datos;
- estructura;
- claves;
- versionado;
- migraciones;
- consistencia;
- recuperación;
- corrupción;
- concurrencia;
- límites;
- exportación;
- importación.

Evalúa si `LocalStorage` realmente es suficiente para todo el proyecto.

Si el tamaño o complejidad lo justifica:

proponer migración progresiva a `IndexedDB`.

No migres automáticamente si rompe demasiadas cosas.

Diseña una estrategia segura:

v1 → almacenamiento actual v2 → IndexedDB v3 → migraciones

Si ya existe lógica de migración, fortalécela.

7. INTEGRIDAD DE DATOS

Implementa validaciones robustas.

Especialmente:

Productos:

- nombre;
- SKU;
- precio;
- costo;
- stock;
- stock mínimo;
- categoría;
- unidad.

Ventas:

- producto válido;
- cantidad válida;
- precio;
- descuento;
- total;
- fecha;
- vendedor;
- caja.

Inventario:

- stock nunca negativo salvo configuración explícita;
- transferencias válidas;
- almacén origen distinto de destino;
- cantidades positivas.

Finanzas:

- cantidades válidas;
- fechas válidas;
- categorías válidas;
- balance consistente.

Usuarios:

- nombres;
 - roles;
 - permisos;
 - estado.
-

8. POS

Haz una auditoría completa del punto de venta.

Verifica:

- agregar producto;
- eliminar producto;
- cambiar cantidades;
- descuentos;
- impuestos si existen;
- total;
- subtotal;
- pago;
- cambio;
- cierre;
- stock;
- kardex;
- caja;
- turno;
- usuario.

Prueba escenarios:

Caso 1

Venta normal.

Caso 2

Venta de múltiples productos.

Caso 3

Cantidad mayor al stock.

Caso 4

Producto inexistente.

Caso 5

Venta con descuento.

Caso 6

Venta con decimal.

Caso 7

Venta cancelada.

Caso 8

Venta después de recargar navegador.

Caso 9

Venta sin conexión.

Caso 10

Venta con almacenamiento corrupto.

Corrige cualquier inconsistencia.

9. INVENTARIO

Verifica completamente:

- almacenes;
- transferencias;
- kardex;
- conteos;
- ajustes;
- kits;
- stock mínimo;
- vencimientos.

Reglas importantes:

Una transferencia debe:

1. validar stock;
2. crear operación;
3. descontar origen;
4. sumar destino;
5. generar kardex;
6. registrar auditoría.

No debe existir la posibilidad de que el stock desaparezca o se duplique por una transferencia parcialmente ejecutada.

10. FINANZAS

Audita:

- flujo de caja;
- libro;
- gastos;
- nómina;
- presupuestos.

Comprueba todas las fórmulas.

Busca:

- errores de redondeo;
- signos incorrectos;
- doble contabilización;
- fechas;
- balances inconsistentes.

Agrega tests unitarios a las funciones financieras críticas.

11. DASHBOARD

El dashboard debe mostrar información real.

No permitir:

- números falsos;
- KPIs hardcodeados;
- gráficos desconectados;
- estadísticas inconsistentes.

Verifica que:

VENTAS

= datos reales de ventas

INGRESOS

= fuentes financieras reales

GASTOS

= gastos registrados

GANANCIA

= regla financiera documentada

Corrige cualquier métrica engañosa.

12. IA

Audita todo el módulo de IA.

Especial atención a:

- API keys;
- proveedor;
- prompts;
- datos enviados;
- límites;
- errores;
- timeout;
- modo offline;
- privacidad.

NUNCA pongas secretos directamente en el código del cliente.

Diseña una abstracción:

AIService

con soporte para:

- provider;
- configuración;
- timeout;
- retry;
- error handling;
- fallback;
- modo demo.

La aplicación debe continuar funcionando aunque la IA no esté disponible.

13. SEGURIDAD

Realiza una auditoría basada en buenas prácticas modernas de aplicaciones web.

Analiza:

- XSS;
- HTML injection;
- DOM injection;
- almacenamiento local;
- exposición de secretos;
- manipulación de datos;
- validación;
- autenticación;
- autorización;
- sesiones;
- 2FA;
- backups;
- importación;
- exportación.

IMPORTANTE:

No afirmes que StockDesk posee seguridad empresarial si la seguridad es únicamente frontend.

Distingue:

Seguridad real

de

Seguridad simulada / local

Por ejemplo:

Si una función llamada "WAF" solo analiza localmente eventos del navegador, documentarla como simulación local.

Nunca falsear capacidades de seguridad.

14. 2FA

Analiza profundamente el supuesto sistema 2FA.

Determina:

- si es criptográficamente real;
- cómo se generan secretos;
- cómo se validan códigos;
- cómo se almacenan;
- recuperación;
- rate limiting.

Si solamente es una simulación:

renombrarla/documentarla correctamente.

Nunca vender como 2FA de nivel servidor algo que realmente no lo sea.

15. AUTENTICACIÓN

Determina si el login actual es:

- real;
- local;
- simulado;
- basado en PIN;
- basado en contraseña;
- simplemente UI.

Diseña una arquitectura de autenticación comercial.

Debe quedar perfectamente documentado:

LOCAL MODE

y

CLOUD MODE

Si no hay backend:

no fingir que existe autenticación multiusuario real.

16. OFFLINE-FIRST

Haz que StockDesk sea realmente resistente al modo offline.

Debe:

- detectar conexión;
- funcionar sin internet;
- no romperse al perder conexión;
- manejar recuperación;
- guardar operaciones;
- sincronizar si existe backend futuro.

Implementa una capa:

NetworkService

con:

- `isOnline()`
 - listeners
 - reconnect
 - fallback
-

17. PWA

Audita y mejora:

- manifest.webmanifest
- service worker
- icons
- splash
- installability
- cache strategy
- offline page
- update flow

Debe ser instalable cuando técnicamente corresponda.

Evita cachear accidentalmente datos sensibles.

Implementa una estrategia de actualización:

Nueva versión disponible.

[Actualizar]

No perder datos locales.

18. VERCEL

Preparar oficialmente el proyecto para Vercel.

Verificar:

- build;
- rutas;
- assets;
- CDN;
- service worker;
- MIME types;

- SPA routing;
- fallback;
- headers;
- caché;
- seguridad.

Crear cuando sea necesario:

```
vercel.json
```

pero solo si realmente hace falta.

No añadir configuración innecesaria.

19. VARIABLES DE ENTORNO

Diseñar:

```
.env.example
```

Nunca incluir secretos reales.

Crear documentación:

```
docs/ENVIRONMENT.md
```

Explicar:

Development Preview Production

y qué variables necesita cada entorno.

Los secretos deben vivir en Vercel Environment Variables, no en Git.

20. LANDING COMERCIAL

StockDesk no debe parecer una demo de desarrollador.

Crear o mejorar una landing comercial profesional.

Debe explicar:

Hero

StockDesk

"Control total de tu negocio en un solo lugar."

Subtítulo:

Inventario + POS + Finanzas + Reportes + IA

CTA:

Probar StockDesk

Ver demostración

Comprar licencia

21. PÁGINA COMERCIAL

Crear una estructura:

/

/app /pricing /features /demo /docs /support /license /privacy /terms

Si la arquitectura actual no permite rutas reales:

adaptarlo al sistema SPA existente.

22. PÁGINA DE CARACTERÍSTICAS

Agrupar características:

Inventario

Punto de venta

Finanzas

IA

Seguridad

Usuarios

Proveedores

Reportes

Personalización

No usar textos exagerados.

Cada característica debe reflejar la realidad del código.

23. SISTEMA DE PLANES

Proponer una estructura comercial.

Por ejemplo:

Starter

- inventario
- POS
- reportes

Professional

- inventario avanzado
- finanzas
- usuarios
- proveedores
- personalización

Business

- todas las funciones
- IA
- backups
- soporte
- funciones avanzadas

No conectar pagos reales todavía a menos que sea necesario.

Primero crear arquitectura preparada.

24. LICENCIAMIENTO

MUY IMPORTANTE.

El repositorio actualmente es público.

Debes investigar y determinar qué licencia existe actualmente.

Si no existe:

crear una estrategia de licencia comercial.

No inventar una licencia legal como si fuera asesoría jurídica.

Preparar:

`LICENSE`

`LICENSE-COMMERCIAL.md`

`docs/LICENSING.md`

Debe quedar claramente definido:

- quién es propietario;
- qué puede hacer el cliente;
- qué no puede hacer;
- redistribución;
- modificación;
- reventa;
- sublicencia;
- ingeniería inversa según jurisdicción;
- garantía;
- responsabilidad;
- soporte.

Usar lenguaje profesional y añadir un aviso de que el texto debe revisarse jurídicamente antes de utilizarse como contrato definitivo.

25. MODELO DE LICENCIA

Diseñar arquitectura para licencias.

Ejemplo conceptual:

licenseKey

customerId

product

edition

issuedAt

expiresAt

status

signature

features

deviceLimit

Debe existir una arquitectura futura para:

- licencia trial;
- licencia permanente;
- suscripción;
- activación;
- desactivación;
- renovación;
- revocación.

PERO:

no crear una falsa licencia criptográfica solo para aparentar seguridad.

Separar:

LOCAL LICENSE

de

SERVER LICENSE.

26. DEMO MODE

Crear un modo DEMO seguro.

La demo debe contener:

- datos ficticios;
- productos ficticios;
- ventas ficticias;
- clientes ficticios;
- proveedores ficticios.

Debe ser imposible confundir los datos demo con datos del usuario.

Botón:

Restaurar demo

27. PROTECCIÓN CONTRA MANIPULACIÓN

Crear una capa de validación de integridad de datos.

Detectar:

- estructuras corruptas;
- versiones incompatibles;
- modificaciones inválidas.

Pero NO afirmar protección absoluta contra manipulación del cliente.

Todo frontend puede ser manipulado.

Documentar correctamente esa limitación.

28. EXPORTACIÓN

Auditar:

JSON

CSV

PDF

BACKUP

IMPORTACIÓN

Debe existir:

- validación;
 - versionado;
 - checksum si corresponde;
 - mensajes de error;
 - confirmación;
 - recuperación ante fallo.
-

29. BACKUPS

Diseñar:

BackupManager

con:

- export;
- import;
- version;
- metadata;
- timestamp;
- validation.

Formato sugerido:

```
{
  "product": "StockDesk",
  "version": "x.x.x",
  "backupVersion": 1,
  "createdAt": "...",
  "checksum": "...",
  "data": {}
}
```

No romper backups antiguos.

30. IMPORTACIÓN SEGURA

Nunca importar datos ciegamente.

Validar:

- versión;
- estructura;

- tipos;
- campos requeridos;
- integridad.

Si algo falla:

cancelar toda la importación.

Nunca dejar la base en estado parcialmente corrupto.

31. ACCESIBILIDAD

Auditar:

- teclado;
- focus;
- labels;
- contraste;
- navegación;
- aria;
- lectores de pantalla;
- tamaños de botones;
- modales;
- formularios.

Especialmente:

- POS;
 - login;
 - tablas;
 - dashboard.
-

32. RESPONSIVE

Probar:

- 320px
- 375px
- 390px
- 414px
- 768px
- 1024px
- 1280px
- 1440px
- 1920px

No permitir:

- overflow horizontal;
 - tablas inutilizables;
 - botones cortados;
 - sidebar roto;
 - modales fuera de pantalla.
-

33. SAFARI / IOS

Prestar especial atención a:

- LocalStorage;
 - PWA;
 - viewport;
 - input;
 - teclado;
 - sticky;
 - 100vh ;
 - impresión;
 - descargas;
 - IndexedDB si se utiliza.
-

34. RENDIMIENTO

Analizar:

- tiempo inicial;
- tamaño de JS;
- cantidad de scripts;
- CDN;
- render;
- eventos;
- listeners;
- DOM.

Buscar:

- scripts innecesarios;
- carga duplicada;
- módulos no utilizados;
- datos renderizados innecesariamente.

No optimizar destruyendo legibilidad.

35. DEPENDENCIAS EXTERNAS

Inventariar todas las dependencias externas.

Incluye:

- Tailwind CDN
- Google Fonts
- APIs
- librerías
- servicios.

Para cada una:

- propósito;
- versión;
- licencia;
- riesgo;
- fallback.

Si algo crítico depende de CDN:

crear una estrategia adecuada para producción.

36. LICENCIAS DE DEPENDENCIAS

Buscar las licencias de todas las dependencias externas.

Crear:

THIRD-PARTY-NOTICES.md

Incluyendo:

- nombre;
- versión;
- licencia;
- URL;
- uso.

No copiar textos legales largos sin comprobar licencia.

37. SEO

La aplicación privada debe protegerse de indexación.

La landing comercial pública debe poder indexarse si esa es la intención.

Separar:

PRODUCT APP

de

MARKETING SITE

si resulta necesario.

Auditar:

- title;
- description;
- canonical;
- Open Graph;
- Twitter cards;
- favicon;
- structured data.

38. BRANDING

Crear una identidad coherente.

Producto:

StockDesk

Posicionamiento:

Sistema moderno de gestión para pequeños y medianos negocios.

Evitar apariencia genérica de proyecto personal.

39. README PROFESIONAL

Reescribir completamente:

README.md

Debe incluir:

1. Qué es StockDesk
2. Problema que resuelve
3. Características
4. Capturas
5. Demo
6. Instalación
7. Uso
8. Arquitectura
9. Configuración
10. Variables de entorno
11. Desarrollo
12. Producción
13. Vercel
14. Seguridad
15. Backup
16. IA
17. Licencia
18. Roadmap
19. Soporte

Crear badges adecuados.

40. DOCUMENTACIÓN

Crear estructura:

```
docs/  
├─ README.md  
├─ ARCHITECTURE.md  
├─ INSTALLATION.md  
├─ DEPLOYMENT.md  
├─ ENVIRONMENT.md  
├─ USER-GUIDE.md  
├─ ADMIN-GUIDE.md  
├─ SECURITY.md  
├─ BACKUPS.md  
├─ AI.md  
├─ API.md  
├─ LICENSING.md  
├─ TROUBLESHOOTING.md  
├─ CHANGELOG.md  
├─ RELEASE.md  
└─ ROADMAP.md
```

41. MANUAL DE USUARIO

Crear un manual completo.

Explicar:

Inicio

Productos

POS

Inventario

Almacenes

Kardex

Finanzas

Dashboard

IA

Usuarios

Proveedores

Notificaciones

Seguridad

Configuración

Backups

Personalización

Exportaciones

42. MANUAL ADMINISTRADOR

Explicar:

- configuración inicial;
- usuarios;

- roles;
 - permisos;
 - cajas;
 - backups;
 - seguridad;
 - restauración;
 - mantenimiento;
 - actualizaciones.
-

43. TROUBLESHOOTING

Crear problemas comunes.

Ejemplo:

"Los datos desaparecieron"

"El POS no abre"

"No puedo imprimir"

"La IA no responde"

"No puedo instalar PWA"

"La aplicación no carga después de actualizar"

"El backup no importa"

"Vercel devuelve 404"

Cada problema debe tener:

Síntoma

Causa

Solución

44. CHANGELOG

Crear changelog profesional:

CHANGELOG.md

Usar:

Added Changed Fixed Security Deprecated Removed

45. VERSIONADO

Crear una estrategia semántica:

MAJOR.MINOR.PATCH

Ejemplo:

1.0.0

Preparar:

- versión interna;
 - versión visible;
 - changelog;
 - release metadata.
-

46. RELEASE CANDIDATE

Antes de considerar el proyecto "Production Ready":

crear:

docs/PRODUCTION-CHECKLIST.md

Debe comprobar:

- [] instalación
- [] navegación
- [] POS
- [] productos
- [] inventario
- [] finanzas
- [] dashboard
- [] seguridad
- [] backups
- [] exportación
- [] responsive
- [] PWA
- [] offline
- [] Vercel
- [] errores

- [] consola
 - [] rendimiento
 - [] documentación
 - [] licencias
-

47. TESTING

Crear una estrategia de pruebas.

Mínimo:

Unit Tests

- cálculos;
- inventario;
- finanzas;
- descuentos;
- conversiones.

Integration Tests

- ventas → inventario;
- ventas → kardex;
- ventas → caja;
- backup → restore.

E2E

- login;
- producto;
- venta;
- reporte.

Si el stack actual no tiene framework de testing:

seleccionar la opción más sencilla y adecuada.

No introducir herramientas gigantes sin necesidad.

48. ERRORES

La consola del navegador debe quedar limpia en condiciones normales.

Buscar:

- ReferenceError;
- TypeError;

- Promise rejection;
- errores de red no manejados;
- listeners duplicados.

Implementar manejo de errores consistente.

49. UX

Auditar todos los estados:

Loading

Empty

Success

Error

Offline

Saving

Saved

Unsaved Changes

Confirmación

Destructive action

No mostrar pantallas vacías sin explicación.

50. CONFIRMACIÓN DE ACCIONES PELIGROSAS

Todas las acciones destructivas deben requerir confirmación:

- borrar producto;
 - eliminar usuario;
 - borrar almacén;
 - restaurar backup;
 - reiniciar app;
 - eliminar datos.
-

51. SISTEMA DE NOTIFICACIONES

Unificar mensajes:

- success
- info
- warning
- error

Nunca depender de `alert()` salvo casos excepcionales.

52. IMPRESIÓN

Auditar tickets.

Verificar:

58mm

80mm

A4

Comprobar:

- márgenes;
 - saltos;
 - logo;
 - QR;
 - barcode;
 - fecha;
 - cajero;
 - cliente.
-

53. DATOS DE DEMOSTRACIÓN

Crear un archivo:

`demo-data.js`

Con datos realistas.

Ejemplo:

Productos:

Laptop

Mouse

Teclado

Monitor

Audífonos

Impresora

Etc.

Ventas

Gastos

Proveedores

Usuarios

54. LEGIBILIDAD DEL CÓDIGO

Todo código nuevo debe ser:

- claro;
- consistente;
- modular;
- comentado solo cuando sea necesario;
- sin hacks;
- sin código duplicado.

No utilizar comentarios engañosos.

55. DOCUMENTACIÓN VS REALIDAD

Después de terminar:

comparar documentación contra implementación.

Crear una tabla:

Función	Documentada	Implementada	Probada	Producción
---------	-------------	--------------	---------	------------

No marcar ☒ si no fue comprobado.

56. AUDITORÍA FINAL DE SEGURIDAD

Realizar una segunda auditoría después de todos los cambios.

Buscar nuevamente:

- XSS
- secretos
- API keys
- DOM injection
- storage issues
- unsafe HTML
- permisos
- import attacks
- malicious backup
- dependency issues

Corregir todo lo corregible.

57. AUDITORÍA COMERCIAL

Responder internamente:

¿Por qué alguien pagaría por StockDesk?

Identifica:

- problema;
- solución;
- ventaja;
- funciones únicas;
- público;
- alternativa frente a Excel;
- alternativa frente a software caro.

Generar:

docs/POSITIONING.md

58. PROPUESTA DE VALOR

Construir una propuesta clara:

StockDesk permite gestionar:

Inventario

Ventas

Finanzas

Usuarios

Proveedores

Reportes

IA

desde una sola aplicación.

Pero solo utilizar funciones verificadas.

59. PREPARACIÓN PARA VENTA

Preparar el proyecto para que pueda venderse como:

Opción A

Software descargable

Opción B

Aplicación web

Opción C

Licencia por negocio

Opción D

Suscripción

Opción E

White-label

Diseñar la arquitectura para que estas opciones sean posibles.

60. WHITE LABEL

Proponer qué elementos deberían ser configurables:

- nombre;
 - logo;
 - colores;
 - moneda;
 - empresa;
 - dominio;
 - tickets;
 - información fiscal.
-

61. SOPORTE

Crear:

`docs/SUPPORT.md`

Definir:

- soporte;
- bugs;
- actualizaciones;
- documentación;
- contacto.

No inventar canales inexistentes.

Usar placeholders cuando sea necesario.

62. PRIVACIDAD

Crear:

`PRIVACY.md`

Debe explicar de manera honesta:

- datos locales;
- datos de IA;
- APIs externas;
- backups;
- cookies;
- almacenamiento;

- analítica si existe.

No inventar cumplimiento legal.

63. TÉRMINOS

Crear borrador:

TERMS.md

Como borrador informativo.

Debe indicar que necesita revisión jurídica antes de utilizarse como contrato definitivo.

64. POLÍTICA DE ACTUALIZACIONES

Diseñar:

- versión;
 - migración;
 - changelog;
 - backup antes de actualizar;
 - rollback.
-

65. ROLLBACK

Crear estrategia.

Antes de actualizar:

1. backup;
2. registrar versión;
3. migrar datos;
4. validar;
5. actualizar.

Si migración falla:

rollback seguro cuando sea posible.

66. GIT

Limpiar el repositorio.

Revisar:

- commits;
- nombres;
- archivos innecesarios;
- secretos;
- `.gitignore`.

Crear/fortalecer:

`.gitignore`

Debe excluir:

`.env`

`.env.local`

logs

temporales

cache

archivos locales

67. GITHUB

Preparar repositorio profesional.

Agregar:

description

topics

README

LICENSE

security policy

issues templates

pull request template

contributing guide si corresponde

Ejemplo:

```
.github/ISSUE_TEMPLATE/
```

```
.github/PULL_REQUEST_TEMPLATE.md
```

```
.github/SECURITY.md
```

68. SECURITY.MD

Crear:

```
.github/SECURITY.md
```

Explicar cómo reportar vulnerabilidades.

No publicar secretos.

69. CI

Configurar GitHub Actions si realmente aporta valor.

Pipeline mínimo:

- validar HTML;
- validar JavaScript;
- comprobar archivos;
- ejecutar tests;
- build;
- detectar errores.

No crear un pipeline complejo innecesariamente.

70. DEPLOYMENT

Crear una guía:

```
docs/DEPLOYMENT.md
```

Debe explicar:

Local

cómo ejecutar.

Preview

cómo probar.

Production

cómo desplegar.

Vercel

cómo conectar GitHub.

Environment Variables

cómo configurarlas.

71. VERCEL CHECKLIST

Comprobar:

- repositorio conectado;
- rama production;
- build;
- variables;
- dominio;
- HTTPS;
- SPA routing;
- service worker;
- caché;
- preview deployment.

No incluir secretos en Git.

72. PRODUCCIÓN REAL

Simular el escenario:

Usuario entra por primera vez.

Instala.

Configura negocio.

Crea producto.

Registra compra.

Realiza venta.

Stock disminuye.

Kardex se actualiza.

Dashboard cambia.

Se genera reporte.

Hace backup.

Cierra.

Abre nuevamente.

Los datos siguen ahí.

Después:

pierde internet.

La aplicación sigue funcionando.

Regresa internet.

La aplicación sigue funcionando.

73. CHAOS TESTING BÁSICO

Probar:

- refresh durante operación;
 - cerrar pestaña;
 - desconectar internet;
 - importar archivo corrupto;
 - borrar datos inesperadamente;
 - almacenamiento lleno si puede simularse;
 - API IA no disponible.
-

74. NO HACER

NO:

- reescribir todo desde cero porque sí;
- introducir React;
- introducir Next.js;
- introducir frameworks innecesarios;
- destruir módulos funcionales;
- fingir seguridad;
- fingir IA;
- inventar APIs;
- inventar backend;
- inventar licencias;
- inventar cumplimiento legal;
- hardcodear secretos;
- eliminar funcionalidades buenas solo por simplificar;
- cambiar la lógica de negocio sin documentarlo.

Primero intenta mejorar la arquitectura existente.

75. CONSERVAR LA TECNOLOGÍA CUANDO SEA RAZONABLE

Preferencia:

HTML5 CSS3 JavaScript moderno

Mantener arquitectura ligera.

Se pueden introducir herramientas de build/testing/deployment cuando tengan un beneficio claro.

No convertir StockDesk innecesariamente en un framework-heavy application.

76. REALIDAD SOBRE SEGURIDAD CLIENT-SIDE

Ten presente:

Una aplicación puramente frontend no puede garantizar:

- secreto absoluto;
- autenticación fuerte;
- autorización confiable;
- protección contra manipulación del cliente.

Cuando una funcionalidad necesite servidor:

diseñar una interfaz para backend futuro.

77. BACKEND FUTURO

Preparar interfaces desacopladas para poder integrar:

Supabase

PostgreSQL

API propia

Cloudflare

Vercel Functions

u otro backend.

Pero NO introducir backend solamente por seguir una moda.

78. API ABSTRACTION

Crear interfaces como:

StorageService

AuthService

AIService

BackupService

ExportService

LicenseService

NotificationService

Esto permitirá reemplazar implementaciones futuras.

79. LICENSING SERVICE

Diseñar:

```
LicenseService
├─ validate()
├─ activate()
├─ deactivate()
├─ getStatus()
└─ getFeatures()
```

La implementación inicial puede ser local/demo.

Dejar preparada la arquitectura para servidor.

80. PRODUCT EDITIONS

Definir internamente:

FREE/DEMO

STARTER

PRO

BUSINESS

ENTERPRISE

Pero no activar ventas reales hasta que exista backend de licencias/pagos adecuado.

81. FEATURE FLAGS

Preparar sistema:

```
FeatureFlags
```

Para poder activar:

AI

AdvancedReports

MultiUser

CloudBackup

Suppliers

etc.

No duplicar código por edición.

82. TELEMETRÍA

Por defecto:

NO añadir tracking invasivo.

Si se incorpora analítica:

debe ser:

- opcional;
 - documentada;
 - respetuosa de privacidad;
 - desactivable si corresponde.
-

83. PRODUCT ANALYTICS

Preparar hooks internos para eventos, pero no enviar datos externos por defecto.

Ejemplos:

product_created

sale_completed

backup_created

settings_changed

84. ERROR LOGGING

Implementar logger central:

```
Logger.info()  
Logger.warn()  
Logger.error()
```

Evitar `console.log()` dispersos en producción.

Los logs de desarrollo pueden mantenerse de forma controlada.

85. DEVELOPMENT MODE

Crear configuración:

development

production

demo

Permitir comportamiento diferente cuando corresponda.

86. CONFIGURATION

Crear configuración central.

No tener:

100 valores hardcodeados repartidos por módulos.

Ejemplo conceptual:

```
AppConfig  
├─ version  
├─ environment  
├─ storage  
├─ ai  
├─ security  
├─ features  
└─ branding
```

87. PRODUCCIÓN

Cuando termines:

ejecuta una auditoría final y produce:

SCORE

Architecture: /10

Security: /10

Performance: /10

UX: /10

PWA: /10

Reliability: /10

Documentation: /10

Commercial readiness: /10

Deployment readiness: /10

Overall: /10

88. REPORTE FINAL

Crear:

PRODUCTION-AUDIT.md

Debe incluir:

Resumen

Problemas encontrados

Problemas críticos

Problemas solucionados

Problemas pendientes

Riesgos

Arquitectura actual

Arquitectura recomendada

Seguridad

Rendimiento

Vercel

Documentación

Licencia

Estado comercial

Próximos pasos

89. TODO LIST

Crear:

`docs/PRODUCTION-TODO.md`

Organizar:

CRITICAL

HIGH

MEDIUM

LOW

DONE

90. REGLA PARA MODIFICAR CÓDIGO

No realices cambios masivos sin verificar.

Proceso:

1. analizar;
2. explicar internamente el problema;
3. modificar;
4. probar;
5. verificar;
6. documentar.

91. GESTIÓN DE CAMBIOS

Agrupar commits conceptualmente.

Ejemplo:

```
chore: production audit baseline
fix: repair inventory consistency
refactor: split finance modules
feat: production backup validation
docs: add deployment documentation
security: harden input sanitization
chore: prepare vercel production
```

No hacer un único commit gigante con todo mezclado.

92. CRITERIO DE TERMINADO

StockDesk SOLO se puede llamar:

PRODUCTION READY

cuando:

- no existen errores críticos conocidos;
- las funcionalidades principales fueron probadas;
- la documentación coincide con el código;
- no existen secretos expuestos;
- Vercel funciona;
- el PWA funciona;
- los backups funcionan;

- la importación está validada;
 - los datos permanecen después de reiniciar;
 - las funcionalidades simuladas están claramente marcadas;
 - la licencia está documentada;
 - la estructura del proyecto es mantenible.
-

93. OBJETIVO COMERCIAL FINAL

El producto debe dar la impresión de:

"Esto es un software que puedo comprar y utilizar."

Y NO:

"Esto es un proyecto de GitHub de un desarrollador."

94. REGLA FUNDAMENTAL DE HONESTIDAD

No declares:

"enterprise security"

"military encryption"

"GDPR compliant"

"bank-grade security"

"real-time cloud"

"AI-powered"

"2FA"

"automatic cloud backup"

u otras afirmaciones comerciales

a menos que el código realmente las respalde.

Cuando exista una simulación:

decir claramente:

"Simulated"

"Local"

"Demo"

"Client-side"

o equivalente.

95. ENTREGABLES FINALES OBLIGATORIOS

Al terminar debes dejar preparados:

```
README.md
LICENSE
LICENSE-COMMERCIAL.md
PRIVACY.md
TERMS.md
CHANGELOG.md
THIRD-PARTY-NOTICES.md

docs/
├─ README.md
├─ ARCHITECTURE.md
├─ INSTALLATION.md
├─ DEPLOYMENT.md
├─ ENVIRONMENT.md
├─ USER-GUIDE.md
├─ ADMIN-GUIDE.md
├─ SECURITY.md
├─ BACKUPS.md
├─ AI.md
├─ LICENSING.md
├─ TROUBLESHOOTING.md
├─ RELEASE.md
├─ ROADMAP.md
├─ PRODUCTION-CHECKLIST.md
├─ PRODUCTION-AUDIT.md
├─ PRODUCTION-TODO.md
├─ POSITIONING.md
└─ SUPPORT.md
```

y cuando corresponda:

```
.env.example
vercel.json
manifest.webmanifest
```

```
service-worker.js
.github/
├─ SECURITY.md
├─ ISSUE_TEMPLATE/
└─ PULL_REQUEST_TEMPLATE.md
```

96. IMPORTANTE SOBRE EL REPOSITORIO PÚBLICO

Antes de considerar StockDesk listo para vender:

investiga el estado actual de:

- LICENSE
- copyright
- ownership
- dependencias
- archivos públicos
- secretos históricos
- información privada
- commits antiguos

Busca también secretos que hayan podido aparecer históricamente en Git.

Si existe un secreto comprometido:

indicar que debe rotarse.

No asumir que borrar un archivo actual elimina el secreto del historial Git.

97. RESULTADO FINAL ESPERADO

Quiero terminar con:

1.

Código limpio.

2.

Arquitectura modular.

3.

Aplicación estable.

4.

PWA funcional.

5.

Vercel listo.

6.

Documentación profesional.

7.

Licencia preparada.

8.

Manual de usuario.

9.

Manual administrativo.

10.

README comercial.

11.

Demo.

12.

Modo producción.

13.

Plan comercial.

14.

Sistema preparado para licencias.

15.

Checklist de producción.

16.

Auditoría de seguridad.

17.

Auditoría de rendimiento.

18.

Auditoría de funcionalidades.

98. PRIMERA ACCIÓN QUE DEBES REALIZAR

NO empieces reescribiendo código.

Primero:

1. inspecciona todo el repositorio;
2. identifica la arquitectura real;
3. identifica las funcionalidades reales;
4. identifica las funcionalidades simuladas;
5. identifica fallos;
6. identifica riesgos;
7. identifica deuda técnica;
8. crea un informe inicial;
9. crea una prioridad de trabajo.

Después comienza la implementación.

99. FORMATO DE PRIORIDAD

Usa:

P0 — BLOQUEA PRODUCCIÓN

Errores críticos.

P1 — MUY IMPORTANTE

Seguridad, datos, ventas, backups, estabilidad.

P2 — IMPORTANTE

UX, arquitectura, documentación.

P3 — MEJORAS

Optimización, detalles.

100. PRINCIPIO FINAL

No quiero solamente que StockDesk "se vea bonito".

Quiero que se convierta en:

un producto real.

Producto que pueda:

- demostrarse;
- desplegarse;
- documentarse;
- mantenerse;
- actualizarse;
- licenciarse;
- venderse;
- instalarse;
- utilizarse diariamente.

Tu criterio final debe ser:

"¿Podría entregar esta aplicación a un cliente que acaba de pagar por ella?"

Si la respuesta es NO:

continúa auditando y corrigiendo.

Si la respuesta es SÍ:

genera el informe final de producción.

FIN DEL MEGA PROMPT.